

bok25

**基
础**

班、计算机组成原理



程序的机器级代码表示



眼熟最基础的汇编语法和助记符即可!



x86的汇编指令格式

AT&A	intel
只能用小写字母	大小写不敏感
第一个为源操作数 第二个为目的操作数	第一个为目的操作数 第二个为源操作数
寄存器前加%,立即数前加\$	不需要加前缀
内存寻址用()	内存寻址用[]
操作码后紧跟字符表明操作数大小,b表示byte,w表示字,l表示双字	操作码后注明byte ptr,word ptr,dword ptr



x86的汇编指令格式

AT&T	Intel	含义
mov \$100, %eax	mov eax, 100	$100 \rightarrow R[ecx]$
mov %eax, %ebx	mov ebx, eax	$R[ecx] \rightarrow R[ebx]$
mov %eax, (%ebx)	mov [ebx], eax	$R[ecx] \rightarrow M[R[ebx]]$
mov %eax, -8(%ebp)	mov [ebp-8], eax	$R[ecx] \rightarrow M[[R[ebp]-8]$
lea 8(%edx, %eax, 2), %eax	lea eax, [edx+eax*2+8]	$R[edx]+R[ecx]*2+8 \rightarrow R[ecx]$
movl %eax, %ebx	mov dword ptr ebx, eax	长度为4字节的 $R[ecx] \rightarrow R[ebx]$



常用指令

指令	说明	举例
mov指令	将第二个操作数复制到第一个操作数，但不能直接从内存复制到内存	mov eax, ebx mov byte ptr [var], 5
push指令	将操作数压入栈，常用于函数调用 ESP 是栈顶，压栈前先将 ESP 的值减4，然后将操作数压入 ESP 指示的地址	push eax push [var]
pop指令	出栈，出栈前先将 ESP 指示的地址中的内容出栈，然后将 ESP 值加4	pop edi pop [ebx]

常用指令

指令	说明	举例
add/sub指令	add指令将两个操作数相加，将结果保存到第一个操作数中 sub指令将两个操作数相减，将结果保存到第一个操作数中	add byte ptr [var], 10 sub eax, 10
inc/dec指令	inc指令将操作数自加1 dec指令将操作数自减1	inc dword ptr [var] dec eax
imul指令	带符号整数乘法指令 如果有两个操作数，则将两个操作数相乘，将结果保存在第一个操作数中，第一个操作数必须为寄存器 如果有三个操作数，则将第二个和第三个操作数相乘，将结果保存在第一个操作数中，第一个操作数必须为寄存器	imul eax, [var] imul esi, edi, 25
idiv指令	带符号整数除法指令，它只有一个操作数，即除数，被除数为edx:eax中的内容，操作结果的商送到eax，余数送到edx	idiv ebx idiv dword ptr [var]
and/or/xor指令	分别是逻辑与、逻辑或、逻辑异或操作 用于操作数的位操作，操作结果放在第一个操作数中	and eax, 0fH xor edx, edx
shl/shr指令	逻辑移位指令，shl为逻辑左移，shr为逻辑右移 第一个操作数表示被操作数，第二个操作数指示移位的位数	shl eax, 1 shr ebx, c1



常用指令

指令	说明	语法
jmp指令	控制转移到label所指示的地址	jmp <label>
jcondition指令	条件转移指令，依据CPU状态字中的一系列条件状态转移	je <label> jne <label> jz <label> jg <label> jge <label> jl <label> jle <label>
cmp/test指令	cmp 指令用于比较两个操作数的值 test 指令对两个操作数进行逐位与运算 这两类指令都不保存操作结果，仅根据运算结果设置CPU状态字中的条件码	cmp <reg>, <reg> test <reg>, <reg>
call/ret指令	分别用于实现子程序的调用及返回	call <label> ret